

Build Spec — IBKR Analysis & Order-Preparation Desk (Python + React)

Paste this whole file into Claude Code (Opus 4.8) as the initial instruction, or save it as `SPEC.md` at the repo root and tell Claude Code: "Read SPEC.md and build it phase by phase." This is a specification, not a script. Follow the phases in order. Do not skip the non-negotiable constraints. Ask me before any destructive action.

0. What we are building (and what we are explicitly NOT building)

A **single-user, locally-run desktop/web application** that connects to **my own Interactive Brokers account** through IBKR's **official TWS API**, streams and analyses market data, runs technical analysis and backtests, and **prepares orders that I confirm one by one** before they are ever sent. It also exports my realised trades for the year in EUR so I can file my Portuguese IRS.

This tool is an **information and order-preparation system**. The human (me) is always the one who pulls the trigger.

Hard constraints — these are non-negotiable and must hold in every phase

1. **Paper trading is the default and only initial mode.** The app connects to the IB Gateway **paper-trading port** out of the box. Live trading is a separate, deliberate, clearly-marked configuration step that I perform myself (see §8). Never default to, or silently enable, live.
2. **No automatic order execution. Ever.** The system may analyse, score, and *prepare* an order, but an order is only transmitted after I explicitly confirm **that specific order**, with its full details shown to me. There is:
 - No "confirm all" / batch-approve button.
 - No "auto-execute" or "trade without asking" mode.
 - No unattended loop that places orders without per-order human confirmation. A confirmation where I cannot see symbol/side/quantity/order-type/price/estimated-cost is not a real confirmation and must not exist.
3. **A hard kill switch** is always reachable: one action that cancels all open/working orders and blocks any new order preparation until I re-enable. It must work even if the UI is janky.
4. **Risk limits are enforced server-side, before an order can even be presented to me:** max order notional, max position size per symbol, max number of open positions, and a max daily realised-loss limit that, once breached, blocks new order preparation for the day.

5. **My credentials never touch this codebase.** The app never stores or transmits my IBKR username/password. Authentication happens in *my* IB Gateway / TWS login window. The app only opens a socket to an already-authenticated Gateway. No secrets hardcoded; any config (ports, host, account id, limits) lives in a local `.env` / `config.toml` that I own and that is git-ignored.
6. **Full audit trail.** Every generated signal, every prepared order, every confirmation (or rejection), and every fill is written to an append-only local journal with timestamps.

If a requested feature would violate any of the above, **stop and tell me** instead of building it.

1. Tech stack (use exactly these unless you flag a concrete problem)

Backend

- Python 3.11+
- `ib_async` (the maintained successor to the now-archived `ib_insync`; or `ib-api-reloaded`), package `ib_async`, $\geq 2.1.0$). **Do NOT use `ib_insync`** — it was archived in March 2024 after the maintainer's passing and is unmaintained.
- `FastAPI` + `uvicorn` for the local API/websocket server.
- `pandas` + `pandas-ta` (or `TA-Lib` if available) for indicators.
- `Backtrader` as the primary backtesting engine (it has native IBKR integration and an event-driven model that mirrors live execution). Structure the code so a VectorBT-based parameter-sweep module can be added later without rewriting strategy logic.
- `SQLite` via `SQLAlchemy` for local persistence (trades, signals, audit journal, config snapshots). Keep a clean repository/DAO layer so Postgres is a drop-in later.
- `structlog` for structured logging; the audit journal is a separate append-only table + file.
- `pydantic` for all config and order/Signal models.

Frontend

- React + TypeScript + Vite.
- **TradingView** `lightweight-charts` for candlesticks + indicator overlays.
- A small, legible component set (your choice; keep dependencies minimal). No HTML `<form>` auto-submits to external endpoints. Talk only to the local backend.

Process model

- Backend talks to **IB Gateway** (preferred over TWS for stability: lighter, supports auto-restart, survives inactivity). Document that the Gateway must be running and logged

in.

2. Connection layer (broker adapter)

Build an `IBKRAdapter` that wraps `ib_async` and is the ONLY place that imports it.

Requirements:

- Connect to `host:port` from config. **Default port = paper**. Standard ports: IB Gateway paper `4002`, IB Gateway live `4001`; TWS paper `7497`, TWS live `7496`. Make the port come from config; default to IB Gateway paper `4002`.
- On startup, call `reqMarketDataType` and implement a graceful fallback chain: real-time → delayed → delayed-frozen. **Market data over the API is "off-platform"**: even symbols that show live in TWS may only return delayed data via the API unless I'm subscribed. The app must work, clearly labelled, on **delayed data** (it will be the default for an unfunded paper account, which only gets delayed L1 top-of-book + historical — no tick-by-tick, no L2 depth). Surface the *actual* data type per symbol in the UI ("REAL-TIME" vs "DELAYED").
- Expose: connect/disconnect, connection-status stream, account summary, positions, open orders, historical bars (`reqHistoricalData`), and a live bar/tick subscription.
- Robust reconnect with backoff. Detect the **weekly re-authentication** drop (2FA accounts must re-auth ~weekly, typically Sunday) and show a clear "Gateway needs re-login" banner rather than silently dying.
- Remember: **one brokerage session per username**. Document that a second IBKR username is needed if I want paper running while a live session is open elsewhere.

Write integration tests that run against a paper Gateway if reachable, and unit tests with the adapter mocked otherwise.

3. Market-data & indicator engine

- A `MarketDataService` that maintains, per watchlist symbol: latest quote, a rolling OHLCV dataframe (configurable timeframe), and a websocket push to the frontend on each update.
- An `IndicatorEngine` (pure functions over a dataframe, fully unit-tested, no broker coupling): SMA, EMA, RSI, MACD (+ signal/hist), Bollinger Bands, ATR, VWAP, plus volume. Each indicator is independently testable with known fixtures.
- The **exact same** `IndicatorEngine` is used by live analysis AND by backtesting — never two implementations of the same indicator.

4. Strategy & signal layer (analysis only — produces signals, never orders)

- A `Strategy` base class: takes the indicator-enriched dataframe and emits **Signals** (`{symbol, direction: BUY|SELL|FLAT, strength, rationale, timestamp, indicator_snapshot}`).
 - Ship 3-4 well-known reference strategies as examples I can study and toggle: RSI mean-reversion, MACD crossover, SMA/EMA crossover, Bollinger breakout. Each is config-parameterised.
 - A Signal is **never** an order. Signals flow into the order-preparation layer (§6) as *suggestions* that I review. The system does not act on a Signal by itself.
 - Make it trivial for me to write my own strategy by subclassing `Strategy`. The app does not give investment advice or decide *for* me; it surfaces what my own rules say.
-

5. Risk engine (pre-trade gate — runs before any order can be shown)

A `RiskEngine` that every prospective order must pass before it can even be presented for confirmation. All limits come from config and are enforced **server-side**:

- `max_order_notional` (e.g. cap per single order)
- `max_position_per_symbol` (shares and/or notional)
- `max_open_positions`
- `max_daily_realised_loss` — once today's realised loss crosses this, **block new order preparation** until the next trading day; show why.
- Mandatory stop-loss option: allow attaching a stop to any prepared order, and a config flag that *requires* a stop before an order can be confirmed.
- The **kill switch**: `POST /kill` cancels all working orders via the adapter and sets a global `trading_blocked = true` flag that the order-preparation layer checks. A separate explicit action clears it. Surface a big red button in the UI.

Every rejection returns a human-readable reason that the UI shows.

6. Order preparation & confirmation gateway (the heart of the safety model)

This is the only path by which an order can reach IBKR. Implement it carefully.

Flow:

1. A prepared order is constructed (from a Signal I chose to act on, or from me entering one manually): `{symbol, side, quantity, order_type (MKT|LMT|STP|STP_LMT), limit_price?, stop_price?, tif}`.
2. It is run through the `RiskEngine`. If it fails, it is **not** shown as confirmable — the failure reason is shown instead.
3. If it passes, the backend computes and returns a **confirmation payload**: symbol, side, quantity, order type, limit/stop prices, current (and data-type-labelled) market price, estimated cost/proceeds, estimated commission if available, resulting position after fill, and which risk checks passed.
4. The frontend shows a **confirmation modal** with all of that. The order is transmitted (`adapter.placeOrder`) **only** on my explicit click on that specific order.
 - In **live** mode, require an extra friction step: I must type the ticker (or "CONFIRM") to enable the send button. Paper mode can use a single explicit click.
5. After transmit, stream order status → fills back to the UI and write everything to the audit journal.

Explicitly forbidden in this layer (do not implement, even if asked later):

- Any endpoint that confirms more than one order in a single action.
- Any flag, schedule, or loop that calls `placeOrder` without a fresh, per-order human confirm.
- Any "remember my confirmation" / "skip confirmation" toggle.

The default, prominent affordance everywhere in the UI is **Prepare order → review → confirm**, never **Trade now**.

7. Backtesting module

- A `Backtest` runner built on Backtrader that consumes the **same** Strategy + IndicatorEngine as live, over historical bars pulled via the adapter (or imported CSV).
- Inputs: strategy + params + symbol(s) + date range + starting cash + commission model + slippage.
- Outputs: equity curve, drawdown curve, trade list, and metrics (total return, CAGR, Sharpe, max drawdown, win rate, profit factor, avg win/loss, exposure). Render in the UI with charts.
- Make commissions and slippage explicit and configurable — a backtest that ignores them lies.

- Provide a clear written caveat in the UI: past backtest performance does not predict live results; day-trading in particular loses money for most participants.

(Optional, scaffold-only for now: a VectorBT parameter-sweep entry point that reuses the same signal logic for fast multi-parameter exploration. Do not block Phase delivery on it.)

8. Going live (build the guard rails, not a shortcut)

- A single config value `TRADING_MODE = "paper" | "live"`, defaulting to `"paper"`.
 - The app must show an unmissable, persistent banner reflecting the current mode (PAPER = neutral/blue; LIVE = red).
 - Switching to live requires: (a) editing the config myself, (b) pointing the port at the live Gateway myself, and (c) an in-app acknowledgement screen that restates the risks and the fact that orders move real money. Do not provide a one-click paper→live toggle.
 - All the §5/§6 protections apply identically in live, plus the typed-confirmation friction.
-

9. Tax / IRS export (Portugal)

- An `export_realised_trades(year)` function and a UI button that produces an **XLSX and CSV** of all realised round-trips for the calendar year: trade date(s), symbol, ISIN if available, side, quantity, buy price, sell price, fees/commissions, gross proceeds, cost basis, and **realised P&L in EUR** (convert non-EUR using the trade-date rate; make the FX source explicit and store the rate used per line).
 - Purpose: feed Portuguese tools like Tax-Wizard / IRSimples, or hand to an accountant for the Anexo G / Anexo J. State clearly in the UI that **the official IBKR Activity Statement / Flex Query remains the source of truth** and this export is a convenience, not tax advice.
-

10. Build order (deliver in phases; each phase must run and be tested before the next)

- **Phase 1 — Scaffold:** repo layout, config/.env handling (git-ignored), logging, SQLite schema, FastAPI skeleton, React+Vite skeleton, mode banner. No broker yet.  when it boots and shows a (mock) dashboard with a clear PAPER banner.

- **Phase 2 — Adapter (paper):** `IBKRAdapter`, connection status, account summary, positions, historical bars, delayed-data fallback, reconnect, re-auth banner. ✅ when it connects to a paper Gateway and shows my (simulated) account + a historical chart.
- **Phase 3 — Live data + indicators + charts:** `MarketDataService`, `IndicatorEngine`, websocket push, lightweight-charts candlesticks with indicator overlays and per-symbol data-type labels. ✅ when a watchlist streams and indicators render.
- **Phase 4 — Strategies + signals feed:** reference strategies, Signal model, signals panel. ✅ when toggling a strategy produces visible, explained signals (and still cannot place orders).
- **Phase 5 — Risk engine + kill switch:** all limits enforced server-side, kill switch wired, rejection reasons surfaced. ✅ when limit breaches and the kill switch demonstrably block action.
- **Phase 6 — Order preparation + confirmation (paper):** the §6 flow end-to-end against paper, single-order confirm only, audit journal writing. ✅ when I can prepare → review → confirm a *paper* order and see the fill + journal entry, with no batch/auto path existing.
- **Phase 7 — Backtesting:** Backtrader runner + metrics + charts + caveats.
- **Phase 8 — Tax export:** the §9 XLSX/CSV exporter.
- **Phase 9 — Live guard rails:** the §8 mode-switch friction and acknowledgement screen. (I will decide if/when to actually run live. Do not push me toward it.)

For each phase: write tests, run them, show me how to run the phase, and summarise what changed.

11. Engineering ground rules for you (Claude Code)

- Ask before deleting files, dropping tables, or any irreversible action.
- Never hardcode secrets, account numbers, or ports; read them from the git-ignored config.
- Keep `ib_async` isolated behind the adapter so the rest of the app is broker-agnostic and testable.
- Prefer small, reviewable commits per phase with clear messages.
- Write a `README.md` covering: IB Gateway setup, enabling API access in Gateway settings, paper vs live ports, the weekly re-auth reality, the second-username note, how to run backend + frontend, and how to run tests.
- If any instruction here conflicts with the **Hard constraints in §0**, the §0 constraints win, and you flag the conflict to me rather than resolving it silently.
- Do not add telemetry, remote logging, or any outbound network call except to the local IB Gateway and (for the optional FX rate in §9) a single explicit, documented rate

source I approve.

12. A note you should keep visible to me in the app

Day trading loses money for the large majority of people who attempt it, independently of how good the tooling is. A solid, regulated broker and a well-built dashboard do not change that — market risk is mine to carry. This app exists to inform my decisions and to make me confirm each one deliberately, on paper first. It is not, and must not become, an autonomous trader.